← Back **Quiz 3**
Graded Assignment • 1h

Due Mar 23, 11:59 PM CET

Your grade: 97.50%

Your latest: **97.50%** • Your highest: **97.50%** • To pass you need at least 80%. We keep your highest score.

Next item →

1. What is the name of the folder where Python looks first for module files? 1 / 1 point

- The user's "home" directory.
- The "site-packages" directory where third-party libraries are installed.
- The Python installation directory.
- The current working directory (CWD).

Correct
This is correct because Python always looks in the current working directory first when trying to import files. This is the default behavior unless explicitly changed.

2. What is the specific name for the task in AI where you categorize text information? 1 / 1 point

- Text Sorting
- Text Regression
- Text Grouping
- Text Classification

Correct
This is correct as it accurately describes the process of categorizing or labeling text into predefined groups or classes.

3. You have a text file, **my_daily_journal.txt** in which you write about how your each day was spent. For each entry, you mention a date. 1 / 1 point

You want the AI Chatbot to give you a summary of your first week of the month of September.

With what you have learned in the course, what is the proper way of reading contents of a file that would come in place of **### ADD MISSING FILE READ CODE HERE ###** ?

You have the following piece of code:

```
### ADD MISSING FILE READ CODE HERE ###

prompt = f"""
Using my daily enteries in my journal (my_journal), summarise everything that
happened during the first week of September 2024.
"""

# Get the response from the LLM
response = get_llm_response(prompt)

# Print the LLM response
print(response)
```

- ```
  f = open("my_daily_journal.txt", "w")
  my_journal = f.readfile()
  f.close()
  ```
- ```
  f = open("my_daily_journal.txt", "w")
  my_journal = f.read()
  f.close()
  ```
- ```
  f = open("my_daily_journal.txt", "r")
  my_journal = f.read()
  f.close()
  ```
- ```
  f = open("my_daily_journal.txt", "r")
  my_journal = f.readfile()
  ```

Correct
This code opens the file in read mode ("r"), reads its content, and explicitly closes the file using f.close(), ensuring proper resource management

4. True or False? Python will automatically close a file after you open it and read its contents? 1 / 1 point

- True
- False

Correct
Python does not automatically close files after read operations; you must close them explicitly using the file.close() command.

5. You have the following sentence stored in a text file, *my_progress.txt* : 1 / 1 point

I have started Module 3 of DeepLearning.AI's AI Python for Beginners.

Think through what each of the following lines of code does:

```
updated_progress = """I completed Module 3 of DeepLearning.AI's AI Python for
Beginners."""

f = open("my_progress.txt", "w")
f.write(updated_progress)
f.close()

f = open("my_progress.txt", "r")
progress = f.read()
f.close()

print(progress)
```

What will appear on screen after you run `print(progress)` ?

- I completed Module 3 of DeepLearning.AI's AI Python for Beginners.
- I have started Module 3 of DeepLearning.AI's AI Python for Beginners.
- Nothing, because the file was never read properly.
- I have started Module 3 of DeepLearning.AI's AI Python for Beginners. I completed Module 3 of DeepLearning.AI's AI Python for Beginners.

Correct
This is the correct answer. When the file "my_progress.txt" is opened in write mode ("w"), the old content is erased, and the new content ("I completed Module 3 of DeepLearning.AI's AI Python for Beginners") is written to the file. Then, the file is opened in read mode ("r"), and the new content is read and printed.

6. You have a set of customer reviews stored in a text file. Each review can be categorized as having either "positive" or "negative" sentiment. You want the AI Chatbot to give you a **total count** of **"positive"** and **"negative"** reviews contained in the file. 1 / 1 point

Which of the following AI Chatbot text prompt is best suited for this task?

- Can you read the reviews and tell me what you think about them?
- The file contains customer reviews. Each review can be categorized as 'positive' or 'negative'. What is the total count of positive and negative reviews?
- Are there more good or bad reviews in the file?
- Please analyze the reviews and give me a summary.

Correct
This prompt is clear and specific. It defines the task (categorizing reviews) and explicitly asks for the total count of each category, ensuring a precise and accurate response.

7. You have a list of destinations in CSV format stored in the variable **places**: 1 / 1 point

```
City,Country
New York,USA
Rio de Janeiro,Brazil
Cape Town,South Africa
Istanbul,Turkey
Paris,France
Tokyo,Japan
Sydney,Australia
```

You want a travel plan of visiting these places, **starting from New York** on **July-01** with **4-day stay** in each city, but you want an itinerary with the **least amount of travel distance**. And you want the output in a table with headings **"Arrival"**, **"Departure"**, **"City"** and **"Country"**.

Which of the following prompts would be best suited for this task to be done by an AI Chatbot?

- prompt = f"""Provide a travel plan for visiting the cities in the list starting from New York on July-01 with a 4-day stay in each city. Output the results in a table format with "Arrival", "Departure", "City" and "Country"."""
- prompt = f"""I have a list of cities I want to visit. Start my trip from New York on July-01 and stay in each city for 4 days. Output the results in a table with "Arrival", "Departure", "City" and "Country"""
- prompt = f"""Plan my travel itinerary starting from New York on July-01 with a 4-day stay in each city. Output the results in a table format with "Arrival", "Departure", "City" and "Country"."""
- prompt = f"""Here is a CSV formatted table of cities I want to travel to (places). Please plan my travel so that I start from New York and have to travel the least amount of distance. My trip should start on July-01 and my stay in each city should be for 4 days. Output the results in table format, with "Arrival" and "Departure" dates, "City" and "Country"."""

Correct
This prompt is detailed and includes all necessary information: the CSV table, the requirement to minimize travel distance, the start date, the duration of stay, and the desired output format.

8. Which of the following is the correct way to implement a Python function that *returns* a miles to kilometers conversion? 1 / 1 point

- ```
  def miles_to_kilometers(miles):
      kilometers = miles * 1.60934
  ```
- ```
  def miles_to_kilometers(miles):
  kilometers = miles * 1.60934
  ```
- ```
  def miles_to_kilometers(miles):
      kilometers = miles * 1.60934
      return kilometers
  ```
- ```
  miles_to_kilometers(miles):
      kilometers = miles * 1.60934
      return kilometers
  ```

Correct
This option correctly defines the function with a colon, includes proper indentation, and has a return statement that returns the kilometers value.

9. Which of the following correctly implements a function in Python to calculate and return the **square** of a number (i.e. the number multiplied by itself)? 0.8 / 1 point

(Select all that apply)

- [x] ```
  def square_number(number):
      number_squared = number * number
      return number_squared
  ```

Correct
This is correct because it properly defines a function "square_number" that takes a single parameter, "number". calculates a new variable called "number_squared" that is equal to the input number multiplied by itself, and then returns the new "number_squared" variable. This is a correct way to define a function in Python that performs an operation with an input parameter and returns a result.

- [ ] ```
  def sqqure_number(number):
      return number_squared
  ```
- [ ] ```
  def square_number(number):
      return number * number
  ```
- [ ] ```
  def square_number():
      return number * number
  ```

You didn't select all the correct answers

10. You used the following piece of code to create a detailed travel itinerary: 1 / 1 point

```
# Create an empty dictionary to store the itinerary for each destination
detailed_itinerary = {}

# Use the "for" loop over the "itinerary" list
for trip_stop in itinerary:
    city = trip_stop["City"]
    country = trip_stop["Country"]
    arrival = trip_stop["Arrival"]
    departure = trip_stop["Departure"]

    rest_dict = read_csv("{city}.csv")

    print(f"Creating detailed itinerary for {city}, {country}.")

    prompt = f"""I will visit {city}, {country} from {arrival} to {departure}.
    Create a daily itinerary with detailed activities.
    Designate times for breakfast, lunch, and dinner.

    I want to visit the restaurants listed in the restaurant dictionary without repeating any place.
    Make sure to mention the speciality that I should try at each of them.

    Restaurant dictionary:
    {rest_dict}

    """
    # Store the detailed itinerary for the city to the dictionary
    detailed_itinerary[city] = get_llm_response(prompt)
```

All the itinerary related files included details of and stops in cities of New York, Rio, Cape Town, Istanbul, Paris, Tokyo and Sydney for 8 days each.

After generating your travel itinerary, your plans changed. Now instead of stopping in Paris, France for 8 days, you'll be stopping there for 4 days, and also, you added another 4 day stop in your itinerary for London, England.

How will you update the existing detailed travel itinerary ? Assume all of the related files have been updated with the new information of London and Paris.

- Remove the entries for Paris and London from the itinerary list, process the remaining cities, and then manually add the updated entries for Paris and London.
- Use an "if" statement within the "for" loop to selectively process changes for Paris and add London, adjusting the itinerary list accordingly.
- Create a new piece of code specifically for London entry and manually edit the output for Paris to reflect the shorter stay.

Correct
This is correct because it leverages the existing code structure to accommodate the changes with minimal adjustments. By using conditional logic ("if" statement) within the loop, the code can selectively process the updated information for Paris and the new data for London efficiently. This approach maintains the script's integrity and adaptability, demonstrating a fundamental principle in programming: modifying data inputs and conditions to achieve desired outputs without unnecessary duplication of code.